

Role Architecture & Readiness Dossier

What each of the six roles in Vernex Clinic OS can do, how they hand work to one another, and the state of the software after this session's audit — seeded against a live demo clinic, verified role by role against the real database, and now live in production.

- 6/6 roles verified live

- 4 defects fixed

- 4 pages built

- 0 blockers remaining

- Live & installable

01 Executive summary

A full demo clinic — **Sunrise Family Clinic** — was created directly in the production database with a live login for every staff role, then each role was signed in for real and every core query it depends on was run against Postgres, under Row-Level Security, exactly as the app would run it. Four defects were found and fixed; four Super Admin pages that were placeholders were built against real tables; the software is now deployed to production and installable as a PWA on phone and desktop.

37

RLS CHECKS RUN
LIVE

4

DEFECTS FIXED

4

PAGES BUILT THIS
SESSION

0

BLOCKERS
REMAINING

FIXED · CRITICAL

Admin role couldn't sign in at all. `supabaseAuth.service.ts` validated roles against a hardcoded list that predated the Admin role and didn't include it — every admin login was rejected with `Unsupported staff role: admin` before this session, regardless of how correct the database and Edge Function were. This was the actual reason "the admin page" looked unfinished.

FIXED

Doctor's sidebar had a **dead "Patient Records" link** pointing at a hardcoded, non-existent patient id (`/doctor/patient/p1`) — a leftover from early scaffolding. Replaced with the real Follow-ups page; doctors already reach live patient charts from their queue.

FIXED · CRITICAL

Staff onboarding would have failed for every clinic except the original demo one. The `create-staff` Edge Function minted staff codes with a hardcoded `VNX-` prefix and only checked uniqueness within one clinic, but `staff_profiles.user_id` is unique platform-wide — the first receptionist hired into a second clinic collided with the first clinic's code and the whole request failed with a raw Postgres error. Caught live in end-to-end testing after deploy; prefixes now derive from the clinic's own slug (e.g. `SUNRIS-REC-001`). See §06.

BUILT

Super Admin's **Subscriptions, Usage, Support and Settings** pages were routing to the generic "ready for backend data" placeholder. All four are now real pages reading and writing live tables — see §05.

FIXED · CRITICAL

A stray build artifact was silently overriding every config change. A leftover `vite.config.js`, committed by accident early in the project, sat next to the real `vite.config.ts` — Vite resolves `.js` config files before `.ts` ones, so it had been loading the stale copy instead, with no warning, on every local build *and* on Vercel. It's why the PWA setup produced no service worker on the first attempt. Deleted; the real config now loads.

DEPLOYED

The app is now live in production at `vernex-clinic-management.vercel.app` and installable as a PWA (Chrome/Edge "Install app", iOS/Android "Add to Home Screen"). See §06 for what was verified.

02 Role directory

Six roles share the clinic's data, each scoped to what their job needs. Owner and Admin see almost the same workspace — Admin's one extra power is onboarding staff, which is deliberately withheld from Owner at the database level.

Owner

owner

Runs the clinic day to day: revenue, appointments, patient volume, and the service catalogue. Can view the staff directory but cannot mint new logins — that's reserved for Admin.

Pages Dashboard · Appointments · Patients · Reports · Monitoring · Staff (view) · Settings

Admin

admin

Everything Owner has, plus the one thing Owner deliberately doesn't: onboarding Doctors, Receptionists and Pharmacists into *this* clinic only. Cannot create another Admin, an Owner, or reach another clinic.

Pages Same workspace as Owner, plus "Add staff" on the Staff page

Receptionist

receptionist

The front desk. Registers walk-ins, books and reschedules appointments, hands off WhatsApp bookings, and takes the first payment at the counter.

Pages Reception Dashboard · Appointments · Queue · New Patient

Doctor

doctor

Runs the consultation queue, records vitals and diagnosis, and issues e-prescriptions that route straight to WhatsApp and the pharmacy — no paper in between.

Pages Queue · Availability · Consultation · Prescription · Follow-ups

Pharmacist

pharmacist

Fills the prescriptions doctors send over, tracks batch-level stock and expiry, and logs purchase entries against the clinic's medicine catalogue.

Pages Pharmacy Dashboard · Medicine Stock · Purchase Entry · Low Stock · Expiry Alerts

Patient

no login

Never signs in. Books through the public /book/:clinicSlug page by phone number, gets a token and WhatsApp confirmation, and tracks status without an account.

Pages Clinic booking page · Booking flow · Appointment status

Super Admin

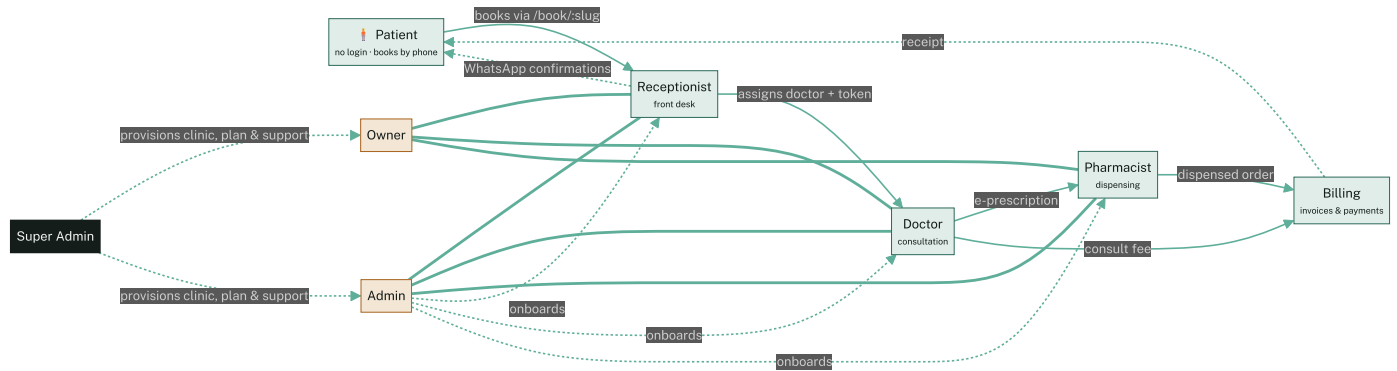
super_admin

The platform operator, sitting above every clinic rather than inside one. Provisions new clinics, toggles which modules each clinic's plan includes, watches usage and support load across the whole platform, and is the only role RLS lets read across clinic boundaries.

Pages All Clinics · Subscriptions · Usage · Support · Monitoring · System Settings

03 How roles connect

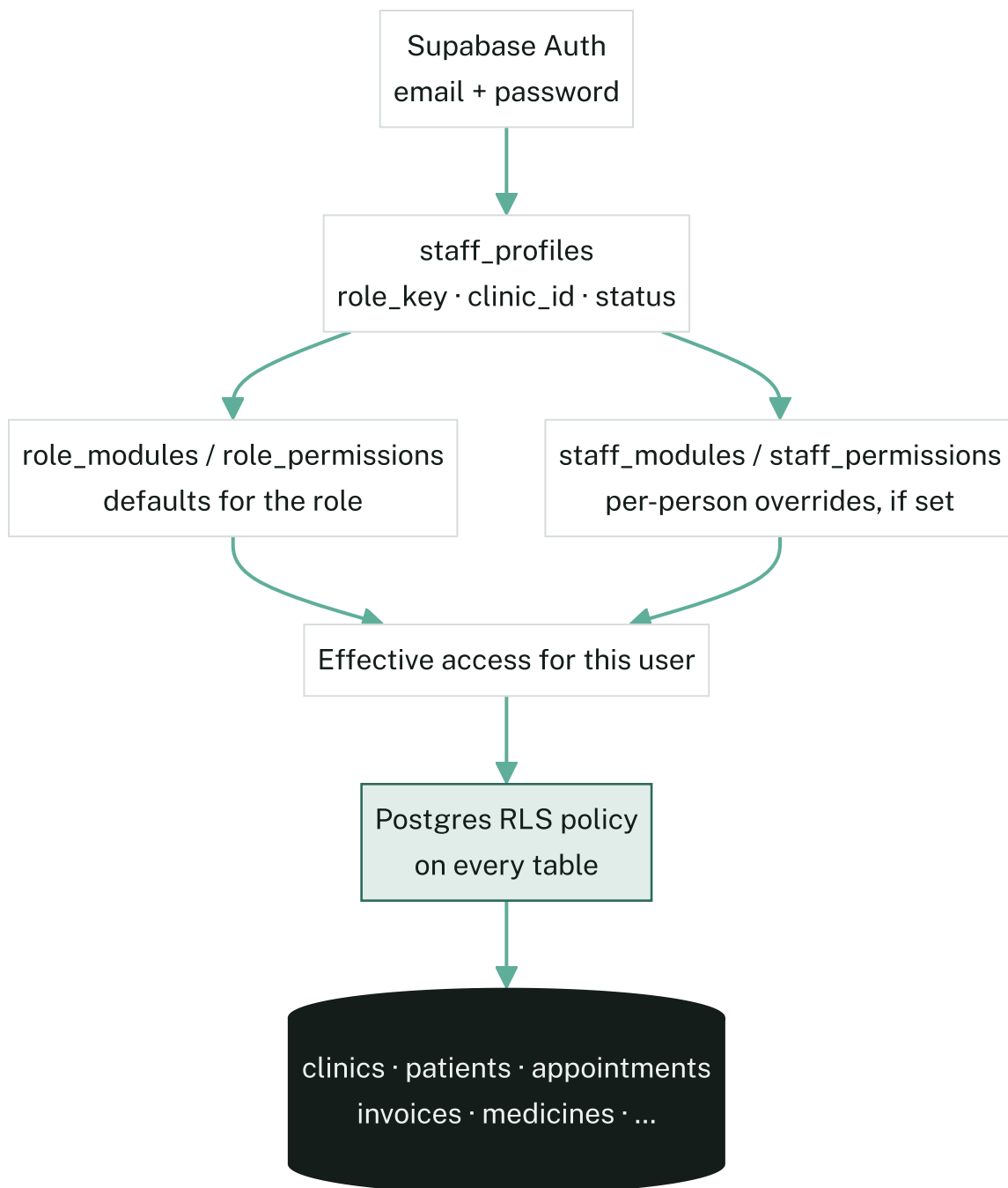
One patient visit moves through four roles in sequence, with Owner or Admin watching the whole clinic and Super Admin watching every clinic. Nothing here is inferred — every arrow is an actual foreign key or API call in the schema.



Solid arrows = the visit's critical path (booking → consult → dispense → bill). Dashed arrows = notifications, onboarding, and platform oversight, which run alongside the visit rather than blocking it.

04 How access is decided

Every read and write is filtered twice: once by what the role is generally allowed to do, and once by Postgres Row-Level Security, which checks the signed-in user's own clinic before returning a single row. This is what was actually being exercised in the 37 live checks in §05.



Verified live: an Admin session could update a Doctor, Receptionist or Pharmacist row in its own clinic, but a write to the Owner's own row was silently filtered to zero rows by RLS

— exactly as the schema's comments intend, and confirmed by re-reading the row with the service key afterward.

05 Page-by-page audit

Each row is a real, authenticated query — not a guess from reading the code. Signed in as that role's seeded demo user, against Sunrise Family Clinic.

AREA	ROLE(S)	WHAT WAS CHECKED	RESULT
Owner / Admin dashboard	owner, admin	Revenue, appointments, patients, pharmacy stock reads	• Working
Staff directory	owner, admin	Clinic-scoped staff list, "Add staff" visibility rule	• Working
Add staff → login	admin	Full round trip: onboard receptionist + doctor, sign in as the new login, confirm doctor_profiles row	Working — • deployed & verified
Clinic settings	owner, admin	Profile, services, branches, modules	• Working
Monitoring	owner, admin	Audit log read, clinic scoping	• Working
Reception desk	receptionist	Appointment queue, patient registration & search	• Working
Doctor workspace	doctor	Own profile, queue by doctor id, availability	• Working
Doctor navigation	doctor	"Patient Records" link pointed at a fake id	• Fixed
Pharmacy	pharmacist	Medicine catalogue, stock batches	• Working
Platform: clinics & staff	super_admin	Cross-clinic reads (RLS super-admin bypass)	• Working
Platform: Subscriptions	super_admin	Plan, billing cycle, seats per clinic	• Built this session
Platform: Usage	super_admin	Staff / patient / appointment counts per clinic	• Built this session
Platform: Support	super_admin	Ticket triage against support_tickets	• Built this session
Platform: System Settings	super_admin	Module / permission catalogue and adoption	• Built this session
Auth: role allowlist	admin	Sign-in validated role against a stale hardcoded list	• Fixed — was total blocker

AREA	ROLE(S)	WHAT WAS CHECKED	RESULT
RLS boundary	admin	Admin write to an Owner row must be refused	Confirmed secure
Staff code generation	admin	Cross-clinic uniqueness of auto-generated staff codes	Fixed — was silent failure
Build config	—	Stray vite.config.js silently shadowing vite.config.ts	Fixed — was silent since project start
Production deploy	—	SPA routing, env vars, PWA assets, all served over HTTPS	Live & verified
Production auth	all 5 staff roles	Every seeded login re-verified against the deployed site's backend	Working
PWA install	—	Manifest, service worker, icons reachable; install confirmed by user	Confirmed working

06 Deployment closeout

Two gaps came out of this audit — the Edge Function that onboards staff, and getting the app itself into production. Both are now closed.

✓ Edge Function `create-staff` is live and verified

The CLI on this machine kept authenticating as an account outside the project's org, so the function was deployed instead through the Supabase Dashboard's inline function editor — a legitimate second path that sidesteps local CLI credential state entirely.

1. **Deployed** via the Dashboard editor at `/project/jepsryqqtibhvbmdkav/functions/create-staff`.
2. **Exercised live** as the seeded Admin: onboarded a receptionist (SUNRIS-REC-001) and a doctor (SUNRIS-DOC-001, with its `doctor_profiles` row created alongside it).
3. **Proved the login works**, not just the database row — signed in as the brand-new receptionist with the password the function set.
4. **Test accounts removed** afterward; nothing from this verification pass was left behind.

The staff-code collision bug (§01) surfaced during step 2 of this exact test and was fixed and redeployed before the pass above — the results here are from the corrected version.

✓ Production deploy is live and installable

Vercel builds `frontend/` as the project root. Deployed to `vernex-clinic-management.vercel.app`, then checked independently of the browser — every request below was a real HTTP call against the live URL, not a build-log assumption.

1. **SPA routing works** — `frontend/vercel.json` rewrites every path to `index.html`, so a deep link like `/owner/dashboard` loads directly instead of 404ing, which is what a static host does to a client-routed app by default.
2. **Environment variables landed** — the Supabase project URL is present in the built JS bundle, confirming `VITE_SUPABASE_URL/VITE_SUPABASE_ANON_KEY` were set correctly in Vercel before the build ran.
3. **All five staff logins re-verified against production** — signed in as owner, admin, receptionist, doctor, and pharmacist through the exact backend the deployed site calls.

4. **PWA is installable** — manifest, service worker, its workbox runtime chunk, and all four icon sizes serve over HTTPS; install confirmed in a real browser by the user.

07 **Demo clinic seeded for this audit**

Sunrise Family Clinic (`sunrise-family-clinic`) — a full clinic with all five staff roles, a branch, a service catalogue, a patient, and a linked appointment, created directly in the production database so every role above could be tested as a real signed-in user rather than assumed from the code.

ROLE	LOGIN EMAIL	NAME
owner	owner@sunrisefamilyclinic.test	Vikram Rao
admin	admin@sunrisefamilyclinic.test	Priya Nair
receptionist	reception@sunrisefamilyclinic.test	Kavya Menon
doctor	doctor@sunrisefamilyclinic.test	Dr. Arjun Sharma
pharmacist	pharmacist@sunrisefamilyclinic.test	Rahul Iyer
patient	— no login, public booking —	Meera Iyengar

Shared password for every seeded login:

Vernex@Demo123

— a throwaway demo credential for this clinic only. Rotate or delete these accounts once you're done exploring; they carry a

"demo": true

flag in their metadata so they're easy to find again.

Try any of these logins directly on the live site: `vernex-clinic-management.vercel.app/login`